

DNP3 Study Guide

Distributed Network Protocol 3

From Serial Substations to Secure Authentication

SCADA Hub Training Series

Ricardo Barbosa

SCADA Hub — barbosaricardo.github.io/dnp3-study-guide

2026

Abstract

DNP3 (Distributed Network Protocol 3) is the dominant SCADA communications protocol in North American electric utilities and water systems. This guide covers the full protocol stack from physical layer framing through Secure Authentication v5. It is written for working field engineers, SCADA integrators, and protection engineers who need to configure, troubleshoot, and certify DNP3 systems — not for academics who want a literature survey. Every section includes concrete field examples, gotchas that have burned real engineers, and the spec-level detail you need to pass interoperability testing.

Certification relevance: ISA CCST (Levels I–III), ISA CAP, NERC reliability standards (FAC, COM), utility device acceptance testing.

Contents

1	Chapter 1: DNP3 Overview	2
1.1	What Is DNP3?	2
1.2	Where DNP3 Is Used	2
1.3	DNP3 vs. The Competition	2
1.4	Key Terminology	3
2	Chapter 2: Protocol Layers	3
2.1	The Three-Layer Stack	3
2.2	Layer Boundaries and Sizes	3
2.3	Physical Connections	4
3	Chapter 3: Data Link Layer	4
3.1	Frame Structure	4
3.2	CRC-16/DNP	4
3.3	The Control Byte	5
3.4	Frame Count Bit (FCB) Mechanism	5

3.5	Addressing	5
4	Chapter 4: Application Layer	5
4.1	APDU Structure	5
4.2	Application Control Byte	6
4.3	Function Codes	6
4.4	Internal Indications (IIN)	6
4.5	Select-Before-Operate (SBO)	7
4.6	Qualifier Codes	7
5	Chapter 5: Data Objects & Groups	8
5.1	The Group/Variation System	8
5.2	Binary Input Quality Flags	8
5.3	Analog Input Quality Flags	9
5.4	Control Relay Output Block (CROB) — Group 12 Var 1	9
5.5	Event Classification	9
6	Chapter 6: Function Codes	9
6.1	READ (FC=0x01)	10
6.2	WRITE (FC=0x02)	10
6.3	SELECT and OPERATE (FC=0x03, 0x04)	10
6.4	FREEZE and FREEZE_AND_CLEAR (FC=0x07, 0x08)	10
6.5	DIRECT_OPERATE (FC=0x05)	10
7	Chapter 7: Unsolicited Responses	11
7.1	What Are Unsolicited Responses?	11
7.2	Startup Sequence for Unsolicited	11
7.3	Unsolicited Hold-Off Timer	11
7.4	Multi-Master and Unsolicited	11
8	Chapter 8: Secure Authentication	12
8.1	Background: Why SA Exists	12
8.2	SA v5 Key Hierarchy	12
8.3	Challenge-Response Flow	12
8.4	Nonce and Replay Protection	12
8.5	Key Change Procedure	13
8.6	SA v5 and NERC CIP	13
9	Chapter 9: Troubleshooting	13
9.1	Bottom-Up Layer Isolation	13
9.2	Common Failure Modes	13
9.2.1	Address mismatch	13
9.2.2	COMM_LOST flag	13
9.2.3	IIN2.3 EVENT_BUFFER_OVERFLOW	13
9.2.4	GPS Week Rollover	13
9.2.5	NAT Timeout on TCP DNP3	14
9.2.6	Crystal Oscillator Drift	14
9.3	Wireshark Filters for DNP3	14

10 Chapter 10: Lab & Practice	14
10.1 Getting Your Hands Dirty	14
10.2 Lab Environment Options	14
10.3 Lab Exercise: OpenDNP3 Master/Outstation	14
10.4 Lab Checklist: Minimum Competency	15
10.5 pydnp3 Common Pitfalls	15
 Appendix A: IIN Quick Reference	 16
 Appendix B: CRC-16/DNP Reference	 16
 Appendix C: Common Address Assignments	 16

Chapter 1: DNP3 Overview

What Is DNP3?

DNP3 was developed in 1993 by Westronic Inc. for the electric utility industry. The problem it solved: electric utilities needed to communicate with remote substations over slow, unreliable serial links (1200–9600 baud) while getting *timestamped events* with millisecond precision. Modbus couldn't do it. IEC 60870-5 was the European answer; Westronic took 60870-5 as the base and extended it significantly for North American conditions.

The result: a 3-layer protocol stack (Application, Transport, Data Link) that supports event buffering, quality flags, 48-bit UTC timestamps, and eventually — after the Aurora Generator Vulnerability in 2007 — cryptographic Secure Authentication.

Key Concept

DNP3 in one sentence: A **Master** polls an **Outstation**, but the Outstation can also push **Unsolicited Reports** when events occur, complete with timestamps down to the millisecond. It is what Modbus would be if it cared about time.

Where DNP3 Is Used

- **Electric utilities** — Substation RTUs, breaker status, meter data, capacitor bank control
- **Water treatment** — Pump stations, tank levels, chlorine dosing, lift stations
- **Oil & gas** — Pipeline monitoring, compressor stations, custody transfer
- **Transportation** — Traffic signals, rail systems, traffic management centers

If you work at a North American utility and your SCADA talks to field RTUs, there is a very high probability those RTUs speak DNP3. The DNP3 Users Group estimates over 30 million DNP3-capable devices are deployed worldwide.

DNP3 vs. The Competition

Protocol	Origin	Primary Use	Events?	Timestamps?
Modbus RTU/TCP	Modicon, 1979	Legacy PLCs, manufacturing	No	No
DNP3	Westronic, 1993	Electric utilities, water SCADA	Yes	48-bit ms UTC
IEC 60870-5	IEC, 1990s	European utilities	Yes	Yes
IEC 61850	IEC, 2003	Modern substations, relays	Yes (GOOSE)	Microsecond

Warning

DNP3 is **not** Modbus with timestamps. The data model, addressing, framing, CRC polynomial, and communication pattern are all different. If you approach DNP3 with Modbus muscle memory, you will suffer. Start fresh.

Key Terminology

Master The controlling device — initiates polls, issues controls. Usually the SCADA server or EMS.

Outstation

The field device — RTU, IED, or intelligent relay. Responds to polls; sends unsolicited reports.

APDU Application Protocol Data Unit — top-layer payload with function code and data objects.

TPDU Transport Protocol Data Unit — fragments large APDUs for the data link layer.

LPDU Link Protocol Data Unit — the actual frame on the wire, with 0x0564 start bytes and CRC-16/DNP.

IIN Internal Indications — 16-bit flag word in every response reporting outstation health.

Group/Var

DNP3 object system — Group = data type, Variation = format/precision.

Class 0/1/2/3

Data classification — Class 0 = static values, Class 1/2/3 = event buffers by priority.

Chapter 2: Protocol Layers

The Three-Layer Stack

DNP3 uses a subset of the OSI model called the **Enhanced Performance Architecture (EPA)**: only three layers, no session or presentation layer. This is a deliberate choice — fewer layers mean less overhead on slow serial links.

Layer	Role
Application (AL)	Function codes, data objects, IIN flags, fragmentation
Transport (TL)	Segments AL messages > 249 bytes into TPDU chunks
Data Link (DL)	Framing, addressing, CRC error detection, flow control

Each layer wraps the layer above it. A large application message gets fragmented by the Transport layer into 249-byte chunks, each chunk gets wrapped in a Data Link frame, and those frames flow over whatever physical medium you're using (serial RS-485, TCP/IP, radio, fiber).

Layer Boundaries and Sizes

- **Data Link frame:** Maximum 292 bytes (18-byte header + up to 274 payload bytes)
- **Transport segment:** Maximum 249 bytes of application data per segment
- **Application fragment:** Maximum 2048 bytes per APDU fragment
- **Maximum APDU via transport:** $64 \times 249 = 15,936$ bytes (64 transport segments)

Spec Depth

The transport header is a single byte. Bit 7 = FIN (this is the last segment), Bit 6 = FIR (this is the first segment), Bits 5:0 = sequence number (0–63). A single-segment message has FIN=1, FIR=1.

Physical Connections

DNP3 is transport-agnostic. The same application layer works over:

- RS-232 / RS-485 serial (1200–115200 baud)
- TCP/IP (IANA-assigned port 20000)
- UDP/IP (also port 20000)
- Fiber, radio, spread-spectrum — anything that moves bytes

TCP DNP3 is now common in substations. The application layer is identical; only the link layer behavior changes slightly (TCP handles framing reliability, so link layer retries are less critical).

Chapter 3: Data Link Layer**Frame Structure**

Every DNP3 Data Link frame begins with the two-byte start sequence 0x05 0x64. This is the sync pattern — if you see these bytes in a serial capture, DNP3 is flowing.

Field	Size	Description
Start Bytes	2 bytes	Always 0x05 0x64
Length	1 byte	Number of bytes following (header + user data, not including CRCs)
Control	1 byte	DIR, PRM, FCB, FCV bits + Function Code nibble
Destination	2 bytes	Little-endian 16-bit address (0–65519 valid; 65535 = broadcast)
Source	2 bytes	Little-endian 16-bit address
CRC	2 bytes	CRC-16/DNP over header bytes
User Data	up to 16-byte blocks, each with 2-byte CRC	Payload from transport layer

CRC-16/DNP

DNP3 uses a non-standard CRC variant: **CRC-16/DNP** (also called CRC-16/M-BUS).

Spec Depth

CRC-16/DNP parameters: polynomial = 0x3D65, initial value = 0xFFFF, RefIn = True, RefOut = True, XorOut = 0xFFFF. This is *not* the same as CRC-16/ANSI or CRC-16/IBM. Implementing the wrong CRC is a common source of interoperability failures during device bring-up.

Every 16-byte block of user data has its own 2-byte CRC appended. This allows partial-frame error detection without waiting for the entire frame — useful on noisy serial lines where burst errors are common.

The Control Byte

The single Control byte encodes the frame direction and function:

- **DIR** (bit 7): 1 = Master-to-Outstation, 0 = Outstation-to-Master
- **PRM** (bit 6): 1 = Primary station (frame initiator), 0 = Secondary
- **FCB** (bit 5): Frame Count Bit — alternates 0/1 for duplicate detection
- **FCV** (bit 4): Frame Count Valid — 1 if FCB is meaningful
- **FC** (bits 3:0): Link Layer Function Code

Pro Tip

Wireshark tip: Filter for `dnp3` to see decoded frames. The Control byte will decode to readable fields. A Control byte of `0xC4` decodes to DIR=1, PRM=1, FCB=0, FCV=0, FC=4 (UNCONFIRMED_USER_DATA).

Frame Count Bit (FCB) Mechanism

The FCB alternates between 0 and 1 on successive confirmed frames. The outstation echoes back the FCB in its response. If the master receives a response with the wrong FCB, it knows a frame was duplicated or lost. This is DNP3's link-layer duplicate detection for confirmed services.

FCB is NOT used with UNCONFIRMED_USER_DATA — the most common frame type for poll responses.

Addressing

- Valid addresses: 0–65519
- 65520–65534: Reserved
- 65535 (0xFFFF): Broadcast address — no response expected
- 0xFFFC (65532): Self-address — outstation reports its own address when polled at this address (some devices)

A common site configuration error: duplicate outstation addresses. Two RTUs with the same address will both respond to polls, colliding on the serial bus. The symptom is CRC errors and random timeouts. Check the address map before calling the manufacturer.

Chapter 4: Application Layer

APDU Structure

The Application Protocol Data Unit (APDU) is the top-level DNP3 payload:

Field	Description
App Control	Sequence number + FIR/FIN fragmentation bits
Function Code	1 byte, defines operation type
IIN (responses only)	2 bytes, 16 internal indication flags
Object Headers	Zero or more Group/Var/Qualifier/Range/Data tuples

Application Control Byte

- **FIR** (bit 7): First fragment
- **FIN** (bit 6): Final fragment
- **CON** (bit 5): Confirmation requested from receiver
- **UNS** (bit 4): Unsolicited message flag
- **SEQ** (bits 3:0): Sequence number 0–15 (wraps)

For multi-fragment responses: first fragment has FIR=1, FIN=0; intermediate fragments have FIR=0, FIN=0; final fragment has FIR=0, FIN=1. A single-fragment response has FIR=1, FIN=1.

Function Codes

Code	Name	Direction / Use
0x00	CONFIRM	M→O or O→M: Acknowledge receipt
0x01	READ	M→O: Poll for data
0x02	WRITE	M→O: Write values (time, event enables)
0x03	SELECT	M→O: First step of SBO control
0x04	OPERATE	M→O: Second step of SBO control
0x05	DIRECT_OPERATE	M→O: Single-step control (no SELECT required)
0x06	DIRECT_OPERATE_NR	M→O: Direct operate, no response
0x07	FREEZE	M→O: Snapshot counters
0x08	FREEZE_AND_CLEAR	M→O: Snapshot then zero counters
0x14	ASSIGN_CLASS	M→O: Set event class assignments
0x17	DELAY_MEASURE	M→O: Measure one-way propagation delay
0x18	RECORD_CURRENT_TIME	M→O: Record arrival timestamp
0x50	RESPONSE	O→M: Normal response to a READ
0x82	UNSOLICITED_RESPONSE	O→M: Event-driven push
0x81	AUTHENTICATE_REQUEST	M→O: SA v5 challenge

Internal Indications (IIN)

Every outstation response includes a 2-byte IIN field. Each bit is a status flag. You need to know all 16.

IIN	Name	Meaning
IIN1.0	BROADCAST	Received a broadcast message
IIN1.1	CLASS1_EVENTS	Class 1 events available
IIN1.2	CLASS2_EVENTS	Class 2 events available
IIN1.3	CLASS3_EVENTS	Class 3 events available
IIN1.4	NEED_TIME	Outstation needs time sync
IIN1.5	LOCAL_CONTROL	At least one point in local control mode
IIN1.6	DEVICE_TROUBLE	Hardware or configuration problem
IIN1.7	DEVICE_RESTART	Device has restarted (cleared on first READ)
IIN2.0	FUNC_NOT_SUPP	Function code not supported
IIN2.1	OBJ_UNKNOWN	Object Group/Variation not recognized
IIN2.2	PARAM_ERROR	Request parameter out of range
IIN2.3	EVENT_BUFFER_OVERFLOW	Events were discarded (buffer full)
IIN2.4	ALREADY_EXECUTING	Control already in progress
IIN2.5	CONFIG_CORRUPT	Device configuration is corrupt
IIN2.6	RESERVED	Reserved, should be 0
IIN2.7	RESERVED	Reserved, should be 0

Warning

IIN1.7 (DEVICE_RESTART): The outstation sets this bit whenever it restarts and it stays set until the master issues a *READ of IIN with the clear bit* or writes to G80V1 (IIN object). If you see IIN1.7 stuck on, the master is not clearing it. This fills the event buffer with restart events over time.

Select-Before-Operate (SBO)

SBO is the two-step control sequence for safety-critical operations (opening breakers, operating switches):

1. Master sends SELECT (FC=0x03) with control object and parameters
2. Outstation echoes back the same SELECT with a status code
3. If status = 0 (Success), master sends OPERATE (FC=0x04) within the SBO timeout
4. Outstation executes the control and responds

If the OPERATE arrives after the SBO timeout (typically 10 seconds), the outstation rejects it with status code 4 (Timeout). The OPERATE parameters must exactly match the SELECT parameters — any byte-level difference and the outstation will reject with status code 2 (No Select).

Qualifier Codes

Qualifier codes appear in object headers and determine how the range is specified:

Qualifier	Name	Range Format
0x00	8-bit Start/Stop	Indices 0–255
0x01	16-bit Start/Stop	Indices 0–65535
0x06	No Range / All	Read all objects of this type
0x07	8-bit Count	Next N objects
0x08	16-bit Count	Next N objects (N up to 65535)
0x17	8-bit Index	Individually addressed, up to 255
0x28	16-bit Index	Individually addressed, up to 65535

Chapter 5: Data Objects & Groups

The Group/Variation System

DNP3 data is organized into typed objects. Every object has a **Group** (what type of data) and a **Variation** (what format/precision/flags).

Group	Type	Common Variations
1	Binary Input (static)	Var 1: packed (no flags), Var 2: with flags
2	Binary Input Event	Var 1: no time, Var 2: absolute time, Var 3: relative time
3	Double-bit Binary Input	Var 1: packed, Var 2: with flags
4	Double-bit Binary Input Event	Var 1: no time, Var 2: absolute time
10	Binary Output (static)	Var 1: packed, Var 2: with flags
11	Binary Output Event	Var 1: no time, Var 2: with time
12	Control Relay Output Block (CROB)	Var 1: the standard control object
20	Binary Counter (static)	Var 1: 32-bit w/flags, Var 5: 32-bit no flags
22	Binary Counter Event	Var 1: 32-bit no time, Var 5: 32-bit with absolute time
30	Analog Input (static)	Var 1: 32-bit int, Var 2: 16-bit int, Var 5: float, Var 6: double
32	Analog Input Event	Var 1: 32-bit no time, Var 3: 32-bit absolute time, Var 5: float
40	Analog Output Status	Var 1: 32-bit int, Var 3: float
41	Analog Output Block	Var 1: 32-bit int, Var 3: float
50	Time and Date	Var 1: absolute timestamp, Var 3: last-recorded time
51	Time and Date CTO	Var 1: absolute CTO (Common Time of Occurrence)
52	Time Delay	Var 2: time delay fine (ms)
60	Class Data	Var 1: Class 0 (static), Var 2/3/4: Class 1/2/3 (events)
70	File Transfer	Var 3: file command, Var 4: file command status
80	Internal Indications	Var 1: IIN register (used to clear DEVICE_RESTART)

Binary Input Quality Flags

Group 1/2 Variation 2 includes a flags byte per point:

- **bit 0 — ONLINE:** 1 = point is online and valid
- **bit 1 — RESTART:** Set after device restart until first poll
- **bit 2 — COMM_LOST:** 1 = communications to field device lost (IED behind RTU)
- **bit 3 — REMOTE_FORCED:** Value overridden by remote command

- **bit 4 — LOCAL_FORCED:** Value overridden locally (keyswitch, SCADA override)
- **bit 5 — CHATTER:** Point is oscillating faster than configured chatter filter
- **bit 6 — STATE:** The actual binary state (0 or 1)

Analog Input Quality Flags

Group 30/32 Variation 1 flags byte:

- **bit 0 — ONLINE**
- **bit 2 — COMM_LOST**
- **bit 3 — REMOTE_FORCED**
- **bit 4 — LOCAL_FORCED**
- **bit 5 — OVERRANGE:** Value exceeds sensor range
- **bit 6 — REFERENCE_ERR:** Reference check failed

Control Relay Output Block (CROB) — Group 12 Var 1

CROB is the standard control object for binary outputs:

Byte	Field	Description
0	ControlCode	See table below
1	Count	Number of times to operate (1–255)
2–5	OnTime	Duration ON in milliseconds (uint32)
6–9	OffTime	Duration OFF in milliseconds (uint32)
10	Status	0 = Success (response only); request = 0

ControlCode byte breakdown: bits 7:6 = TCC (Trip/Close Code), bit 5 = CR (Clear), bit 4 = QU (Queue), bits 3:0 = Code.

Common Code values: 0x03 = LATCH_ON, 0x04 = LATCH_OFF, 0x41 = PULSE_ON (TCC=01, Code=01).

Warning

If a CROB OPERATE is rejected with status 2 (No Select), byte-compare the SELECT and OPERATE payloads. Even a 1-bit difference in any CROB field (including reserved bits) will cause rejection. This is spec-required behavior, not a device bug.

Event Classification

Points are assigned to event classes (Class 1, 2, or 3) in the Device Profile Document. Class assignment determines event priority:

- **Class 1:** Highest priority — protection trips, critical alarms
- **Class 2:** Medium priority — switching events, major alarms
- **Class 3:** Low priority — equipment status, minor alarms
- **Class 0:** No events — static data only (poll Class 0 to read current values)

Chapter 6: Function Codes

READ (FC=0x01)

The most common function. A Class Poll reads event buffers:

READ: G60V2 (Class 1 events) + G60V3 (Class 2) + G60V4 (Class 3)

READ: G60V1 (Class 0 = all static data)

The outstation responds with a RESPONSE (FC=0x81) containing whatever objects match the request. If events are pending, IIN1.1–1.3 will be set.

WRITE (FC=0x02)

Used to set time (G50V1 or G50V3), write IIN (G80V1 to clear DEVICE_RESTART), assign event classes (with ASSIGN_CLASS FC=0x14), and set unsolicited configuration.

Time sync procedure:

1. Master sends DELAY_MEASURE (FC=0x17) to measure propagation delay
2. Outstation responds with G52V2 (delay in ms)
3. Master sends WRITE of G50V1 with current time + half measured delay
4. Outstation clears IIN1.4 (NEED_TIME)

SELECT and OPERATE (FC=0x03, 0x04)

Covered in Chapter 4 SBO section. Key additional points:

- The SELECT initiates a timeout timer on the outstation (typically 10s, device-configurable)
- Only one SBO sequence can be active per outstation at a time
- A new SELECT cancels any pending SELECT
- If the master gets no response to SELECT, retry before concluding failure

FREEZE and FREEZE_AND_CLEAR (FC=0x07, 0x08)

These apply to counters (Group 20):

- **FREEZE:** Copies current counter values to frozen registers (Group 21). Non-destructive.
- **FREEZE_AND_CLEAR:** Copies to frozen registers, then zeroes the live counters. The snapshot and zero happen atomically — a counter increment during this operation is deterministically included in the frozen value or the new running value, never lost.

Spec Depth

FREEZE_AND_CLEAR atomicity is spec-required (IEEE 1815 clause 4.3.4). If a vendor implementation is non-atomic, energy meter billing calculations will accumulate errors over time. Test this with a pulse source during a FREEZE_AND_CLEAR if billing accuracy matters.

DIRECT_OPERATE (FC=0x05)

Single-step control, no SELECT required. Faster than SBO but provides no protection against race conditions or spurious controls. Use only where SBO is not required by the system design, usually for non-critical outputs (annunciators, auxiliary relays).

Chapter 7: Unsolicited Responses

What Are Unsolicited Responses?

An outstation sends an Unsolicited Response (FC=0x82) when events occur without waiting for a poll. This is the event-driven alternative to polling:

- Master enables unsolicited responses by writing to the outstation's configuration
- Outstation sends events as they occur (or batched up to a configurable count/time threshold)
- Master must confirm each unsolicited response (FC=0x00 CONFIRM)
- If no CONFIRM is received, outstation retries up to 3 times, then holds the buffer (does not discard)

Key Concept

An unconfirmed unsolicited response causes the outstation to **hold** its event buffer — not discard. Events are retained until confirmed. This prevents data loss on noisy links at the cost of buffer memory.

Startup Sequence for Unsolicited

Multi-master environments require a careful startup sequence:

1. Master sends DISABLE_UNSOLICITED (FC=0x15) to prevent spurious messages during initialization
2. Master polls Class 0 (full static scan) to get initial state
3. Master polls Class 1/2/3 to drain any buffered events
4. Master sends ENABLE_UNSOLICITED (FC=0x14 with appropriate class objects)
5. Outstation begins sending events without being polled

Skipping step 1 causes an outstation to send buffered events during startup before the master is ready to process them. The events are not lost, but they confuse state machines that assume initial scan is the baseline.

Unsolicited Hold-Off Timer

The hold-off timer (also called the startup wait time) delays the outstation from sending unsolicited reports after a power cycle. This prevents event storms when a substation powers up and every IED sends events simultaneously.

A well-configured system staggers these timers: RTU 1 = 30s, RTU 2 = 45s, RTU 3 = 60s. Poorly configured systems all fire at 0s and saturate the communication channel.

Multi-Master and Unsolicited

Only **one master** should have unsolicited enabled per outstation. The second master polls at a slower rate. If both masters enable unsolicited, each gets a stream of events and each must CONFIRM them — but the outstation tracks which master confirmed which sequence number. This works technically but doubles confirmation traffic and complicates troubleshooting.

Chapter 8: Secure Authentication

Background: Why SA Exists

In 2007, the Aurora Generator Vulnerability demonstrated that an attacker with access to a SCADA communication channel could issue control commands — breaker trips — and cause physical damage to generators. DNP3 at the time had zero authentication. Any device that could speak DNP3 could issue arbitrary commands.

SA v2 (Challenge-Response with HMAC-SHA-1) was added to the spec. SA v5 (the current version, part of IEEE 1815-2012) extended it with bidirectional authentication and session key management.

SA v5 Key Hierarchy

Update Key (UK) Long-lived symmetric key pre-provisioned on master and outstation. Never transmitted on the wire. Used only to derive session keys. Changed via the Key Change Procedure.

Session Keys Short-lived keys derived from the Update Key. Separate keys for each direction: Monitor Direction (M→O) and Control Direction (O→M). Rotated periodically (typically every 15–60 minutes or after N messages).

HMAC The authentication MAC. SA v5 uses HMAC-SHA-256 truncated to 80 bits (10-byte MACs).

Challenge-Response Flow

1. Outstation issues a Challenge (random nonce, sequence number, key status)
2. Master computes HMAC over the challenge + the message it wants to send
3. Master sends AUTHENTICATE_REQUEST (FC=0x20) with the HMAC
4. Outstation verifies; if valid, processes the original message
5. On failure after retries: critical operations are blocked

Aggressive Mode: Rather than waiting for a challenge, the master pre-emptively sends a MAC with every critical message. If the outstation accepts aggressive mode, it skips the challenge round-trip. This halves the authentication overhead on low-latency links.

Nonce and Replay Protection

Nonces are cryptographically random values that prevent replay attacks. SA v5 requires a CSPRNG (Cryptographically Secure Pseudo-Random Number Generator) — not a simple counter. An attacker capturing an authenticated frame cannot replay it because the nonce will never repeat.

Warning

If your outstation uses a sequential counter as a nonce instead of a CSPRNG, it is technically non-compliant with SA v5 and may be vulnerable to replay attacks. This is a known implementation quality issue in some older devices. Check the device profile document.

Key Change Procedure

The Update Key itself must be changed periodically (NERC CIP and good practice). The Key Change Procedure (FC=0x21) uses AES Key Wrap (RFC 3394) to encrypt the new Update Key under the existing one. The session must be authenticated before a Key Change can proceed.

SA v5 and NERC CIP

NERC CIP-007 (Systems Security Management) and CIP-005 (Electronic Security Perimeters) do not require DNP3 SA specifically, but they do require authentication for SCADA communications at many utilities. SA v5 is the accepted technical implementation for DNP3. If your utility is NERC CIP-applicable, plan for SA v5 on any DNP3 link that crosses a security perimeter.

Chapter 9: Troubleshooting

Bottom-Up Layer Isolation

Always troubleshoot from the physical layer up. Chasing application layer errors when the CRC failure rate is 20% wastes everyone's time.

1. **Physical:** Check cable wiring, termination resistors (RS-485: 120Ω at each end), signal levels, baud rate match, parity match.
2. **Data Link:** Capture frames. Zero CRC errors? FCB sequencing correct? Correct addresses?
3. **Transport:** Are fragments being assembled? FIR/FIN sequence correct? No missing segments?
4. **Application:** Function codes correct? IIN flags? Object types match device profile?

Common Failure Modes

Address mismatch — Master polls address 5; device thinks it's address 6. Symptom: no responses, ever. Device is silently ignoring the frames. Fix: verify address in device configuration.

COMM_LOST flag — If an RTU polls a downstream IED (relay behind the RTU) and that IED goes offline, the RTU sets COMM_LOST on all points from that IED. The master sees the quality flag and treats the value as stale. This is correct behavior — the RTU is telling you it can't verify the data.

IIN2.3 EVENT_BUFFER_OVERFLOW — The outstation's event buffer filled up and events were discarded. Causes: poll rate too slow, communication channel down for extended period, event storm from a chattering contact. Recovery: issue a full Class 0 poll to resynchronize state. Events before the overflow are gone.

GPS Week Rollover — DNP3 timestamps are 48-bit milliseconds since 1970-01-01 UTC — they do NOT use GPS week numbers. But many RTUs get their time from a GPS receiver that uses GPS time internally. GPS week rollovers (last occurred April 6, 2019) can corrupt the GPS receiver's output. The symptom is timestamps that are 19.7 years off. If you see timestamps from early 2000 in 2026, GPS week rollover is your first suspect.

NAT Timeout on TCP DNP3 — TCP DNP3 sessions over NAT firewalls will be dropped by the NAT table if no traffic flows for the NAT idle timeout (often 5–15 minutes). The DNP3 master’s keepalive poll must be more frequent than the NAT timeout. Or configure the firewall to extend the timeout for TCP port 20000.

Crystal Oscillator Drift — RTU clocks drift without GPS synchronization. The drift is temperature-dependent and follows a quadratic curve — not linear. A 20 ppm crystal at 25°C might drift 50 ppm at 65°C. After 24 hours without GPS sync at 20 ppm: $20 \times 10^{-6} \times 86400\text{s} = 1.7\text{s}$ drift. Events near protection relay operation will have inaccurate sequence of events records. Always verify GPS sync status before using timestamps for post-event analysis.

Wireshark Filters for DNP3

```
tcp.port == 20000      # All TCP DNP3 traffic
dnp3                  # DNP3 protocol layer only
dnp3.iin.broadcast == 1 # Frames with BROADCAST set
dnp3.al.func == 0x82    # Unsolicited responses only
dnp3.al.func == 0x03    # SELECT commands
```

Save your capture file before making any configuration changes. You will want to compare before/after.

Chapter 10: Lab & Practice

Getting Your Hands Dirty

Reading about DNP3 is useful. Actually building a master, watching it poll an outstation, and seeing event buffers fill up in real time is where it becomes permanent knowledge.

Lab Environment Options

OpenDNP3 C++ DNP3 library by Automatak. Free, Apache 2.0. Full SA v5 support. The gold standard for open-source DNP3. Ships with demo programs for master and outstation.

pydnp3 Python bindings for OpenDNP3. Write a DNP3 master or outstation in Python. Great for scripting and testing.

Triangle MicroWorks Test Harness Commercial. The industry standard for interoperability testing. If your device passes TMW, it works.

Wireshark Free. Decodes DNP3 out of the box. For TCP DNP3, filter `tcp.port == 20000`.

Lab Exercise: OpenDNP3 Master/Outstation

```
# Build OpenDNP3 with demos
git clone https://github.com/automatak/dnp3.git
cd dnp3 && mkdir build && cd build
cmake .. -DDEMO=ON && make -j4
```

```
# Terminal 1: start outstation (listens on 127.0.0.1:20000)
./outstation-demo

# Terminal 2: start master
./master-demo

# Terminal 3: capture in Wireshark
# Filter: tcp.port == 20000
```

Watch the console. You will see Class 1/2/3 polls, responses, and unsolicited events. Capture in Wireshark and decode each frame type manually to verify your frame structure understanding.

Lab Checklist: Minimum Competency

Before claiming DNP3 competency, you should be able to:

- Run an outstation simulator and connect a master to it
- Capture the startup exchange in Wireshark and decode each frame type
- Identify all 16 IIN bits in a response
- Enable and disable unsolicited responses and observe the difference
- Trigger a binary input change and watch the event flow from buffer to master
- Decode a DNP3 object header: Group, Variation, Qualifier, Range
- Perform a Select-Before-Operate and observe both request/response pairs
- Force an event buffer overflow and observe IIN2.3 being set
- Sync time using Group 50 Variation 1
- Read a Device Profile Document and configure a master poll schedule from it

pydnp3 Common Pitfalls

1. **Not calling `master.Enable()`:** The stack is built but communication never starts.
2. **Process exit without `manager.shutdown()`:** The IO service thread hangs. The process appears frozen on exit.
3. **Ignoring `ISOEHandler.process_header()`:** This is where data arrives. If you don't implement it, you receive nothing.
4. **asyncio + pydnp3:** pydnp3 runs its own IO thread. Mixing it with asyncio `asyncio.run()` requires careful thread boundary management.

Appendix A: IIN Quick Reference

Bit	Flag	Action Required
IIN1.0	BROADCAST	Informational — no action required
IIN1.1	CLASS1_EVENTS	Poll G60V2 to retrieve events
IIN1.2	CLASS2_EVENTS	Poll G60V3 to retrieve events
IIN1.3	CLASS3_EVENTS	Poll G60V4 to retrieve events
IIN1.4	NEED_TIME	Send DELAY_MEASURE + WRITE G50V1
IIN1.5	LOCAL_CONTROL	Investigate which points are in local mode
IIN1.6	DEVICE_TROUBLE	Check device hardware and logs
IIN1.7	DEVICE_RESTART	Write G80V1 with clear bit to acknowledge
IIN2.0	FUNC_NOT_SUPP	Check master poll configuration vs device profile
IIN2.1	OBJ_UNKNOWN	Check Group/Variation vs device profile
IIN2.2	PARAM_ERROR	Check qualifier and range in the failing request
IIN2.3	EVENT_BUFFER_OVERFLOW	Increase poll frequency or event buffer size
IIN2.4	ALREADY_EXECUTING	Wait for previous control to complete
IIN2.5	CONFIG_CORRUPT	Contact device vendor

Appendix B: CRC-16/DNP Reference

Parameter	Value
Polynomial	0x3D65
Initial Value	0xFFFF
Input Reflection (RefIn)	True
Output Reflection (RefOut)	True
Final XOR	0xFFFF
Residue	0x66C5
Check Value (for “123456789”)	0xEA82

A 5-byte header with CRC 0x0000 always fails. If your library returns 0x0000 for any input, the XorOut is not being applied.

Appendix C: Common Address Assignments

Address	Use
0	Often used for Master address (not standardized)
1–65519	Valid outstation addresses
65520–65534	Reserved
65532 (0xFFFFC)	Self-address (outstation responds with its own address)
65535 (0xFFFFF)	Broadcast — no response expected